

Diseño Rápido de Interfaces Gráficas

JavaFX + FXML

con Agentes de Codificación (OpenCode)



Asignatura: Diseño de Interfaces (DI)

Curso: 2024-2025

Autor: Daniel García

Objetivo: Manual de usuario para acelerar el diseño de interfaces JavaFX desde bocetos/mockups usando herramientas de IA gratuitas

Índice

- 1. Introducción y Objetivos**
- 2. Requisitos Previos**
- 3. Visión General del Flujo de Trabajo**
- 4. Etapa 1: De Mockup/Boceto a Descripción Digital**
 - a. Alternativa A: Foto + IA con Visión
 - b. Alternativa B: Herramientas de Wireframing
 - c. Alternativa C: Descripción Textual Directa
 - d. Alternativa D: Scene Builder como Referencia
 - e. Comparativa de Alternativas
- 5. Etapa 2: De Descripción Digital a FXML con OpenCode**
 - a. Configuración de OpenCode con Modelos Gratuitos
 - b. Fichero de Instrucciones para el Agente
 - c. Generación del FXML
 - d. Generación del Controller y App
- 6. Scripts de Automatización (Python)**
 - a. Script de Análisis de Mockup
 - b. Pipeline Completo Automatizado
- 7. Ejemplo Práctico Completo**
 - a. Pantalla de Login
 - b. Panel Principal (Dashboard)
- 8. Prompts Utilizados (Referencia Completa)**
- 9. Proyecto JavaFX: Ejecución**
- 10. Conclusiones**

1. Introducción y Objetivos

Problema

El diseño de interfaces gráficas en JavaFX usando FXML es un proceso que tradicionalmente requiere:

- Conocimiento detallado de la sintaxis XML de FXML
- Familiaridad con los layouts y componentes de JavaFX
- Tiempo considerable en Scene Builder arrastrando componentes
- Escritura manual de código repetitivo (Controllers, imports, fx:id...)

Solución propuesta

Un **toolkit/metodología** que permite a un estudiante pasar de una idea (boceto en papel, wireframe digital, o descripción verbal) a una interfaz JavaFX funcional **lanzable** (sin lógica de negocio) en el menor tiempo posible, usando:

- **IAs con visión** (Gemini, ChatGPT) para analizar bocetos
- **Agentes de codificación** (OpenCode) para generar FXML
- **Modelos gratuitos** (Gemini Flash, Codestral) para no tener coste
- **Scripts Python** opcionales para automatizar el pipeline

Objetivo del manual

Que cualquier estudiante pueda replicar el proceso y generar interfaces JavaFX rápidamente siguiendo los pasos documentados, sin necesidad de conocimientos avanzados de FXML.

2. Requisitos Previos

Software necesario

Software	Versión mínima	Descarga	Obligatorio
Java JDK	17+	adoptium.net	Sí
Apache Maven	3.8+	maven.apache.org	Sí
OpenCode	Última	github.com/opencode-ai/opencode	Sí
Python	3.8+	python.org	Opcional (scripts)
Scene Builder	20+	gluonhq.com	Opcional (verificar)

API Keys necesarias (gratuitas)

Google Gemini API Key (Recomendado):

- 1. Ir a <https://aistudio.google.com/apikey>
- 2. Iniciar sesión con cuenta de Google
- 3. Pulsar "Create API Key"
- 4. Copiar la key generada

Límite gratuito: ~1500 peticiones/día con Gemini 2.0 Flash

Instalación de dependencias Python (opcional)

```
pip install google-generativeai Pillow
```

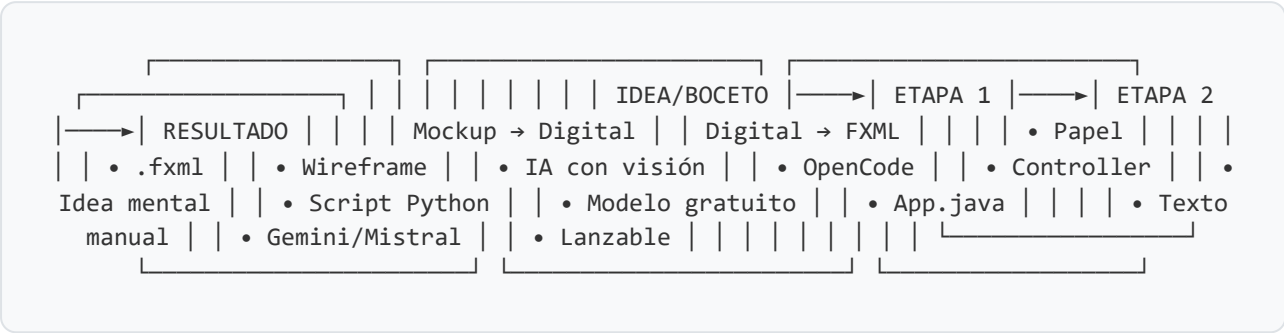
Configuración de la API Key

```
# Windows (CMD)
set GOOGLE_API_KEY=tu-api-key-aqui

# Windows (PowerShell)
$env:GOOGLE_API_KEY="tu-api-key-aqui"

# Linux/Mac
export GOOGLE_API_KEY="tu-api-key-aqui"
```

3. Visión General del Flujo de Trabajo



Tres opciones de flujo según velocidad

Opción	Flujo	Velocidad	Automatización
A: Script Python	Foto → Script → Todo generado automáticamente	Máxima	Total
B: OpenCode guiado	Descripción .md → OpenCode → FXML + Java	Alta	Semi-auto
C: Solo OpenCode	Descripción verbal directa a OpenCode	Alta	Manual con agente

4. Etapa 1: De Mockup/Boceto a Descripción Digital

El objetivo de esta etapa es convertir una **idea visual** (boceto en papel, wireframe, o idea mental) en un **fichero de texto estructurado** que un agente de codificación pueda interpretar correctamente.

4.a) Alternativa A: Foto + IA con Visión (Recomendada)

La forma más rápida y natural de trabajar.

Herramientas gratuitas con visión:

Herramienta	URL	Calidad
Google Gemini	gemini.google.com	Excelente
ChatGPT (GPT-4o mini)	chatgpt.com	Muy buena
Microsoft Copilot	copilot.microsoft.com	Muy buena

Proceso:

- 1 Dibujar el boceto en papel (no necesita ser perfecto, solo claro)
- 2 Hacer una foto con el móvil (buena iluminación, sin sombras)
- 3 Subir la foto a una IA con visión (Gemini recomendado)
- 4 Usar el prompt de análisis (ver sección 8)
- 5 Copiar el resultado y guardarlo como fichero `.md`

Prompt para analizar la foto:

Analiza esta imagen de un mockup/boceto de interfaz gráfica de usuario.

Genera una descripción estructurada en formato Markdown orientada a JavaFX/FXML con:

1. LAYOUT GENERAL: Tipo de layout principal (BorderPane, VBox, HBox, GridPane, etc.)
2. COMPONENTES: Lista cada componente visible con:
 - Tipo JavaFX (Button, Label, TextField, TableView, MenuBar, etc.)
 - Texto/contenido visible
 - Posición aproximada (arriba, centro, izquierda, etc.)
 - Tamaño relativo (ancho completo, mitad, etc.)
3. JERARQUÍA: Estructura anidada de contenedores y componentes
4. ESTILOS: Colores, alineaciones, espaciados notables

Formato de salida: descripción técnica orientada a JavaFX/FXML.
Incluye una tabla con los fx:id de cada componente interactivo.

Ventajas: Muy rápido, natural, no requiere herramientas extra de diseño.

Desventajas: Depende de la calidad del boceto y de la interpretación de la IA.

4.b) Alternativa B: Herramientas de Wireframing

Herramientas gratuitas:

Herramienta	URL	Mejor para
Excalidraw	excalidraw.com	Bocetos rápidos estilo mano alzada
Figma	figma.com	Wireframes profesionales
draw.io	app.diagrams.net	Diagramas de layout
Pencil Project	pencil.evolus.vn	Prototipado GUI (open source)

Proceso:

1. Crear el wireframe en la herramienta elegida
2. Exportar como imagen (PNG/JPG)
3. Usar la misma técnica de la Alternativa A (subir a IA con visión)

Ventajas: Diseño más limpio y preciso, colaborativo (Figma).

Desventajas: Requiere aprender la herramienta, más lento que papel.

4.c) Alternativa C: Descripción Textual Directa

Si ya tienes claro lo que quieres, puedes escribir la descripción directamente sin necesidad de imagen.

Plantilla de descripción:

```
# Interfaz: [Nombre de la pantalla]

## Layout principal: [BorderPane/VBox/HBox/GridPane/AnchorPane/StackPane]
## Dimensiones: [ancho x alto en px]

## Estructura:
- **TOP**:
  - [Componente]: [descripción]
- **LEFT**:
  - [Componente]: [descripción]
- **CENTER**:
  - [Componente]: [descripción]
- **BOTTOM**:
  - [Componente]: [descripción]

## Componentes con fx:id:


| ID         | Tipo      | Texto            | Posición |
|------------|-----------|------------------|----------|
| btnLogin   | Button    | "Iniciar Sesión" | CENTER   |
| txtUsuario | TextField | prompt: "User"   | CENTER   |



## Estilos:
- Color primario: #3498db
- Fuente: System 14px
- Espaciado: 10px entre elementos
```

Ventajas: No necesita ninguna herramienta, máximo control, el agente lo interpreta perfectamente.

Desventajas: Requiere conocer los nombres de componentes JavaFX.

4.d) Alternativa D: Scene Builder como Referencia

Usar Scene Builder para crear un prototipo visual rápido y luego dejar que el agente mejore el FXML generado.

Proceso:

1. Abrir Scene Builder
2. Arrastrar componentes básicos (sin preocuparse por estilos)
3. Guardar el FXML
4. Pedir al agente que lo mejore/complete

Nota: Esta alternativa es la menos eficiente porque ya estás haciendo manualmente parte del trabajo que el agente podría hacer. Solo úsala si quieres un punto de partida visual muy concreto.

4.e) Comparativa de Alternativas

Criterio	A: Foto+IA	B: Wireframe	C: Texto	D: SceneBuilder
Velocidad	★★★★★	★★★★☆	★★★★☆	★★★☆☆
Precisión	★★★★☆	★★★★☆	★★★★★	★★★★★
Facilidad de uso	★★★★★	★★★★☆	★★★★☆	★★★★☆
Coste	Gratis	Gratis	Gratis	Gratis
Requiere IA	Sí (visión)	Opcional	No	No
Mejor para	Prototipado rápido	Trabajo en equipo	Control total	Ajustes finos

Recomendación: Para máxima velocidad usa la Alternativa A (foto + IA). Para máximo control usa la Alternativa C (descripción textual). Puedes combinar ambas: foto → IA → ajustar la descripción manualmente → pasar a Etapa 2.

5. Etapa 2: De Descripción Digital a FXML con OpenCode

5.a) Configuración de OpenCode con Modelos Gratuitos

Qué es OpenCode

OpenCode es un **agente de codificación en terminal** (CLI) que permite interactuar con modelos de IA para generar, editar y refactorizar código directamente en tu proyecto.

Instalación

```
# Instalar OpenCode (consultar documentación actualizada)
npm install -g @anthropic-ai/opencode

# Verificar instalación
opencode --version
```

Configuración con Google Gemini (Gratuito)

```
# 1. Obtener API Key en https://aistudio.google.com/apikey

# 2. Configurar variable de entorno
export GOOGLE_API_KEY="tu-api-key"

# 3. Configurar OpenCode para usar Gemini
# (consultar documentación de OpenCode para la configuración específica)
```

Modelos gratuitos recomendados:

Modelo	Proveedor	Calidad para FXML	Velocidad
gemini-2.0-flash	Google	★★★★☆	Muy rápido
gemini-2.5-pro	Google	★★★★★	Medio (límite diario)
codestral-latest	Mistral	★★★★☆	Rápido

5.b) Fichero de Instrucciones para el Agente

El fichero `OPENCODE-INSTRUCTIONS.md` es un fichero que se coloca en la raíz del proyecto para que OpenCode lo lea como contexto. Contiene todas las reglas y patrones que el agente debe seguir al generar FXML.

Cómo usarlo: Al abrir OpenCode en el proyecto, el agente puede leer este fichero con un simple comando: `"Lee OPENCODE-INSTRUCTIONS.md para entender tu rol"`. A partir de ahí, seguirá las reglas definidas.

Contenido clave del fichero de instrucciones:

- Rol del agente (generador de interfaces JavaFX)
- Flujo de trabajo paso a paso
- Reglas obligatorias para el FXML (imports, fx:id, controller, estilos)
- Plantilla de descripción estructurada
- Tabla de referencia de componentes JavaFX
- Convenciones de nombrado

5.c) Generación del FXML

Proceso con OpenCode:

1 Abrir OpenCode en la raíz del proyecto

```
cd mi-proyecto-javafx
opencode
```

2 Pedir que lea las instrucciones

```
Lee el fichero OPENCODE-INSTRUCTIONS.md para entender tu rol.
Luego lee mockups/descripcion-login.md y genera el FXML correspondiente
en src/main/resources/fxml/login.fxml
```

3 Verificar el resultado

- Abrir el FXML en Scene Builder para verificar visualmente
- O ejecutar con `mvn javafx:run`

4 Corregir si es necesario

```
Lee src/main/resources/fxml/login.fxml y corrige:
- Los botones no están centrados
- Falta padding en el contenedor principal
- El campo de contraseña debería ser PasswordField
```

Reglas que el FXML generado debe cumplir:

- Incluir **todos** los `<?import?>` necesarios
- Usar `fx:controller="com.ejemplo.NombreController"`
- Asignar `fx:id` a todos los componentes interactivos
- Convención de `fx:id`: `btn` , `txt` , `lbl` , `tbl` , `col` , `chk` , `cmb`
- Estilos inline con `style="-fx-..."`

- Layout responsive con `hgrow` / `vgrow`

5.d) Generación del Controller y App

Prompt para generar el Controller:

```
Lee el fichero src/main/resources/fxml/login.fxml y genera la clase
Controller Java correspondiente.
- Package: com.ejemplo
- Nombre: LoginController
- @FXML para cada fx:id
- Método initialize() vacío
- SIN lógica de negocio
Guardar en src/main/java/com/ejemplo/LoginController.java
```

Prompt para generar App.java:

```
Genera una clase App.java que extienda Application, cargue
/fxml/login.fxml y muestre la ventana.
Package: com.ejemplo
Guardar en src/main/java/com/ejemplo/App.java
```


6. Scripts de Automatización (Python)

Se incluyen dos scripts Python que automatizan las etapas usando la API gratuita de Google Gemini.

6.a) Script: analizar_mockup.py

Función: Toma una imagen de mockup y genera la descripción estructurada en formato .md

Uso:

```
# Instalar dependencias
pip install google-generativeai Pillow

# Configurar API Key
export GOOGLE_API_KEY="tu-api-key"

# Ejecutar
python scripts/analizar_mockup.py mockups/mi-boceto.jpg Login
```

Resultado:

Genera el fichero `mockups/descripcion-login.md` con la descripción estructurada lista para la Etapa 2.

6.b) Script: pipeline_completo.py

Función: Pipeline completo automatizado. Un solo comando genera todo: descripción + FXML + Controller + App.java

Uso:

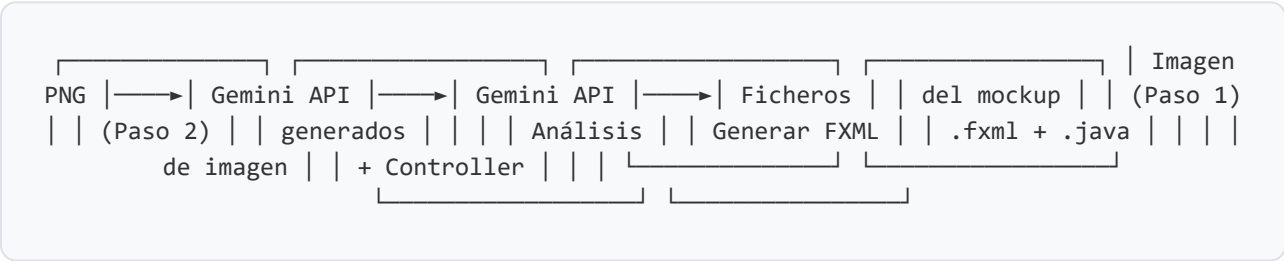
```
# Pipeline completo
python scripts/pipeline_completo.py foto-mockup.jpg NombrePantalla com.mipaquete
```

Qué genera:

Fichero	Ruta
Descripción .md	mockups/descripcion-nombrepantalla.md
FXML	src/main/resources/fxml/nombrepantalla.fxml
Controller	src/main/java/com/.../NombrePantallaController.java
App lanzadora	src/main/java/com/.../App.java

Ventaja principal: De una foto a un proyecto JavaFX lanzable en un solo comando.

Flujo interno del script:



7. Ejemplo Práctico Completo

A continuación se muestra un ejemplo completo con dos interfaces: **Login** y **Panel Principal**.

7.a) Pantalla de Login

Mockup ASCII (simulando un boceto en papel):

(fondo gris)

(tarjeta blanca)

Iniciar Sesión
Accede a tu cuenta

Correo electrónico

Contraseña

☐ Recordarme ¿Olvidaste?

INICIAR SESIÓN

— o —

Crear cuenta nueva

(botón azul)

(botón outline)

Descripción estructurada generada:

```
# Interfaz: Pantalla de Login

## Layout principal: StackPane > VBox (centrado)
## Dimensiones: 400 x 550 px
## Fondo: #f0f2f5

## Estructura:
- StackPane (fondo gris)
  - VBox (centrado, maxWidth 340)
    - VBox (tarjeta blanca, sombra, bordes redondeados, padding 40px)
      - Label "Iniciar Sesión" (24px, bold)
      - Label "Accede a tu cuenta" (13px, gris)
      - TextField (promptText: "Correo electrónico")
      - PasswordField (promptText: "Contraseña")
      - HBox:
        - CheckBox "Recordarme"
        - Region (espaciador)
        - Hyperlink "¿Olvidaste tu contraseña?"
      - Button "Iniciar Sesión" (azul, ancho completo)
      - HBox (separador con "o")
      - Button "Crear cuenta nueva" (outline, ancho completo)
```

FXML generado (fragmento):

```
<?xml version="1.0" encoding="UTF-8"?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>
<?import javafx.geometry.Insets?>

<StackPane xmlns:fx="http://javafx.com/fxml/1"
    fx:controller="com.ejemplo.LoginController"
    prefWidth="400" prefHeight="550"
    style="-fx-background-color: #f0f2f5;">

    <VBox alignment="CENTER" spacing="0" maxWidth="340">
        <VBox alignment="CENTER" spacing="8"
            style="-fx-background-color: white;
                -fx-background-radius: 10;
                -fx-effect: dropshadow(gaussian, rgba(0,0,0,0.15), 15, 0, 0,
3);">

            <padding><Insets top="40" right="35" bottom="35" left="35"/></padding>

            <Label fx:id="lblTitulo" text="Iniciar Sesión"
                style="-fx-font-size: 24px; -fx-font-weight: bold;">

            <TextField fx:id="txtEmail" promptText="Correo electrónico"
                prefHeight="40" maxWidth="Infinity"/>

            <PasswordField fx:id="txtPassword" promptText="Contraseña"
                prefHeight="40" maxWidth="Infinity"/>

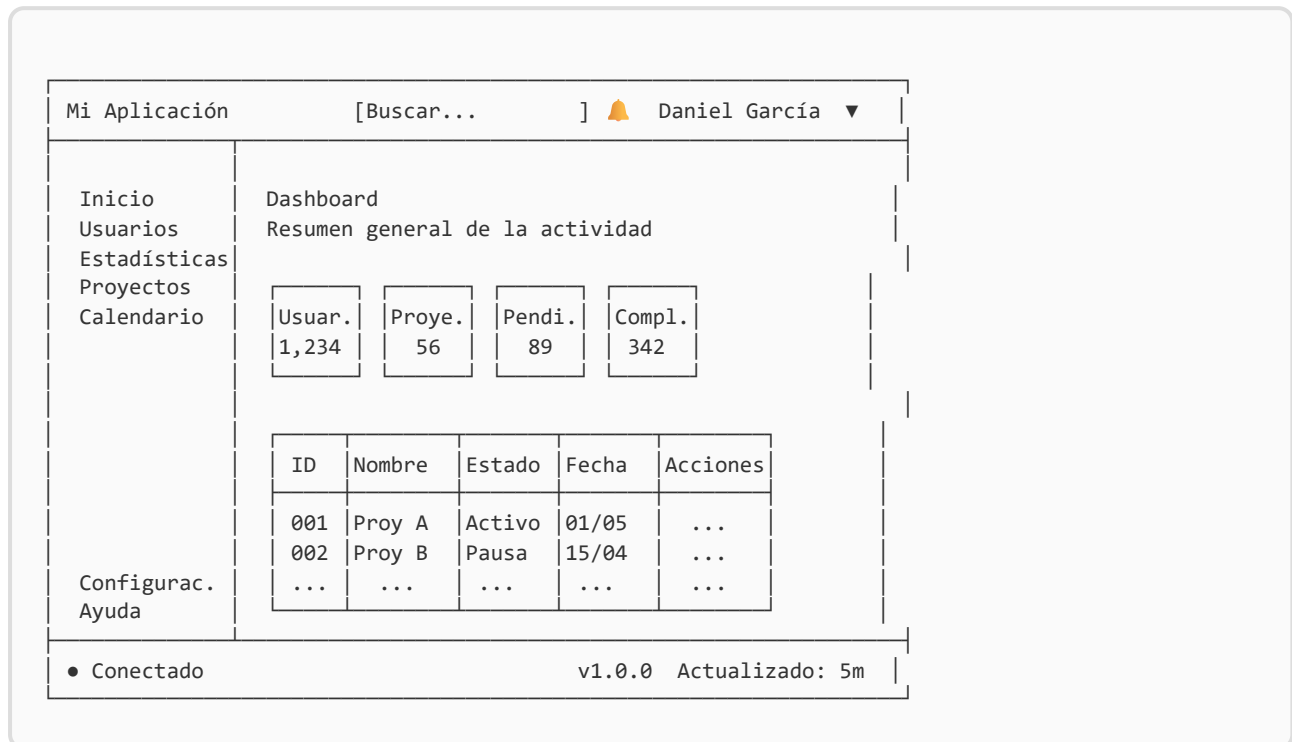
            <Button fx:id="btnLogin" text="Iniciar Sesión"
                prefHeight="42" maxWidth="Infinity"
                style="-fx-background-color: #3498db;
                    -fx-text-fill: white;
                    -fx-font-weight: bold;">

            <Button fx:id="btnRegistro" text="Crear cuenta nueva"
                prefHeight="42" maxWidth="Infinity"
                style="-fx-border-color: #3498db;
                    -fx-text-fill: #3498db;">

        </VBox>
    </VBox>
</StackPane>
```

7.b) Panel Principal (Dashboard)

Mockup ASCII:



Estructura del FXML generado:

- **BorderPane** como layout raíz (900x600px)
- **TOP:** HBox con barra de navegación oscura (#2c3e50)
 - Label (nombre app), Region (espaciador), TextField (búsqueda), Button (notificaciones), MenuButton (usuario)
- **LEFT:** VBox con menú lateral (#34495e)
 - Buttons de navegación (Inicio, Usuarios, etc.)
 - Region (espaciador flexible)
 - Buttons inferiores (Configuración, Ayuda)
- **CENTER:** VBox con contenido principal (#ecf0f1)
 - Labels de título
 - HBox con 4 tarjetas de estadísticas (VBox cada una)
 - TableView con 5 columnas
- **BOTTOM:** HBox con barra de estado

8. Prompts Utilizados (Referencia Completa)

Todos los prompts están guardados en la carpeta `prompts/` del proyecto. A continuación se resumen:

Prompt 01: Analizar mockup con IA de visión

Uso: Subir foto de boceto a Gemini/ChatGPT/Copilot

Resultado: Descripción estructurada en Markdown

```
Analiza esta imagen de un mockup/boceto de interfaz gráfica.  
Genera una descripción estructurada en Markdown orientada a JavaFX/FXML.  
Identifica: layout principal, todos los componentes (tipo JavaFX exacto),  
posiciones, textos, y genera una tabla con los fx:id.  
Usa convención: btn, txt, lbl, tbl, col, chk, cmb.
```

Prompt 02: Generar FXML desde descripción

Uso: En OpenCode, cuando ya tienes el fichero .md de descripción

```
Lee el fichero mockups/descripcion-[nombre].md y genera un fichero FXML  
para JavaFX que implemente esa interfaz.
```

Requisitos:

- Incluir todos los `<?import?>` necesarios
- `fx:controller="com.ejemplo.[Nombre]Controller"`
- `fx:id` en TODOS los componentes interactivos
- Estilos inline con `style="-fx-..."`
- Layout responsive (`hgrow`, `vgrow`, `maxWidth="Infinity"`)
- SIN lógica, solo interfaz visual
- Guardar en `src/main/resources/fxml/[nombre].fxml`

Prompt 03: Corregir/refinar FXML

Uso: En OpenCode, para iterar sobre un FXML ya generado

```
Lee src/main/resources/fxml/[nombre].fxml y corrige/mejora:  
- [Describir el problema concreto]  
- [Ej: "los botones no están centrados"]  
- [Ej: "falta padding", "la tabla no ocupa todo el ancho"]  
Mantén el resto del diseño igual.
```

Prompt 04: Generar Controller y App.java

Uso: En OpenCode, después de tener el FXML

```
Lee TODOS los .fxml en src/main/resources/fxml/ y para cada uno:  
1. Genera su Controller Java vacío con todos los @FXML  
2. Guárdalo en src/main/java/com/ejemplo/[Nombre]Controller.java  
  
Genera también App.java que cargue [nombre].fxml como pantalla principal.  
Package: com.ejemplo
```

Prompt 05: Múltiples pantallas de una vez

Uso: Generar varias interfaces en una sola sesión de OpenCode

```
Genera interfaces FXML para estas pantallas:  
1. LOGIN: [descripción]  
2. REGISTRO: [descripción]  
3. PANEL PRINCIPAL: [descripción]  
  
Para cada una: .fxml + Controller vacío.  
Estilo visual coherente. Misma paleta de colores.  
Al final, App.java que lance login.fxml.
```


9. Proyecto JavaFX: Ejecución

Estructura del proyecto Maven

```
proyecto-javafx/  
├─ pom.xml  
└─ src/main/  
    ├─ java/com/ejemplo/  
    │   ├─ App.java           ← Clase lanzadora  
    │   ├─ LoginController.java ← Controller (vacío)  
    │   └─ PanelPrincipalController.java  
    └─ resources/fxml/  
        ├─ login.fxml        ← Interfaz de login  
        └─ panel-principal.fxml ← Interfaz del dashboard
```

Cómo ejecutar

```
# 1. Navegar al directorio del proyecto  
cd proyecto-javafx  
  
# 2. Compilar y ejecutar con Maven  
mvn javafx:run
```

Cambiar la interfaz que se lanza

En `App.java` , cambiar la constante:

```
// Para ver el login:  
private static final String FXML_A_CARGAR = "/fxml/login.fxml";  
  
// Para ver el panel principal:  
private static final String FXML_A_CARGAR = "/fxml/panel-principal.fxml";
```

pom.xml (dependencias)

```
<dependencies>
  <dependency>
    <groupId>org.openjfx</groupId>
    <artifactId>javafx-controls</artifactId>
    <version>17.0.2</version>
  </dependency>
  <dependency>
    <groupId>org.openjfx</groupId>
    <artifactId>javafx-fxml</artifactId>
    <version>17.0.2</version>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.openjfx</groupId>
      <artifactId>javafx-maven-plugin</artifactId>
      <version>0.0.8</version>
      <configuration>
        <mainClass>com.ejemplo.App</mainClass>
      </configuration>
    </plugin>
  </plugins>
</build>
```

Importante: Se necesita Java 17+ y Maven instalados. Si usas un IDE como IntelliJ o Eclipse, puedes importar el proyecto como proyecto Maven y ejecutar directamente la clase App.

10. Conclusiones

Resumen de lo conseguido

Este toolkit demuestra que es posible **reducir drásticamente** el tiempo de diseño de interfaces JavaFX usando agentes de codificación y herramientas de IA gratuitas:

Aspecto	Método Tradicional	Con este Toolkit
De idea a FXML	Lento (Scene Builder manual)	Rápido (un prompt)
Conocimiento requerido	FXML, layouts JavaFX	Mínimo (la IA lo genera)
Controller boilerplate	Escribir manualmente	Generado automáticamente
Coste	Gratis	Gratis (APIs gratuitas)
Calidad resultado	Alta (manual)	Buena (iterando con agente)

Flujo más rápido posible

1. **Foto del boceto** en papel → subir a Gemini → descripción `.md`
2. **Abrir OpenCode** → pedir que lea la descripción → genera `.fxml`
3. **Pedir Controller** → genera `.java` → ejecutar con `mvn javafx:run`

O con el **script Python**: un solo comando que hace todo.

Herramientas utilizadas

- **OpenCode** - Agente de codificación en terminal (obligatorio)
- **Google Gemini** - Modelo gratuito con visión y generación de código
- **Python** - Scripts de automatización (opcional)
- **Maven + JavaFX 17** - Proyecto ejecutable
- **Scene Builder** - Verificación visual (opcional)

Puntos clave para la defensa

1. **Práctico y reutilizable:** cualquier estudiante puede usar este toolkit
2. **Múltiples alternativas documentadas:** 4 formas de hacer la Etapa 1
3. **Todo gratuito:** no requiere licencias ni suscripciones de pago
4. **Ejemplo funcional:** el proyecto compila y se ejecuta
5. **Prompts documentados:** todos los prompts están guardados y explicados
6. **Automatizable:** scripts Python para el pipeline completo
7. **Extensible:** fácil añadir más pantallas siguiendo el mismo flujo

Manual generado para la asignatura de Diseño de Interfaces (DI) - Subida de Nota

Herramientas utilizadas: OpenCode + Modelos gratuitos (Gemini)